# Introduction to Basic and Advanced Statistical Modelling

## Using R

Alexandre Courtiol

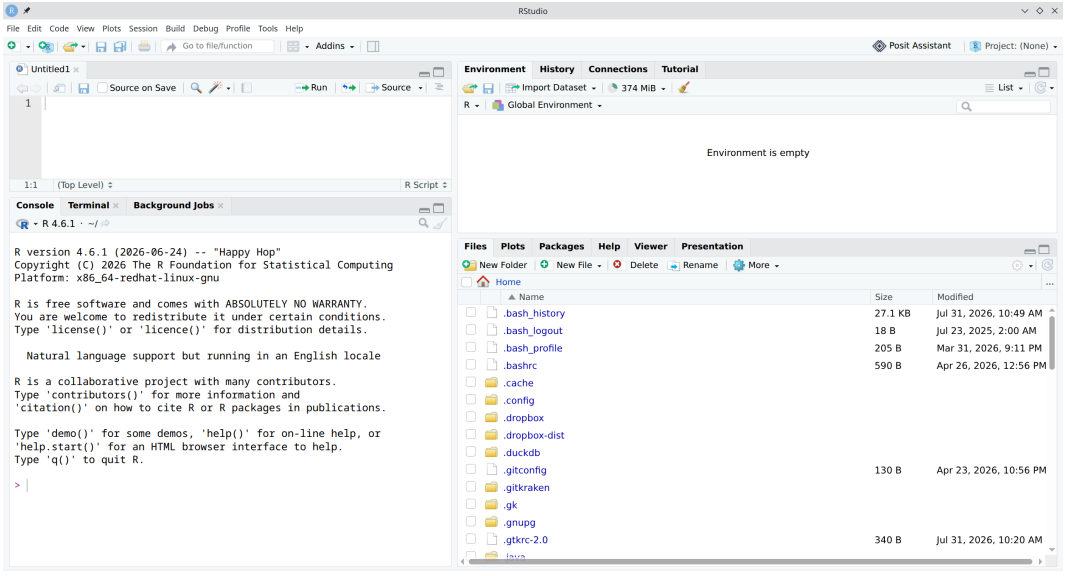
# Table of contents

# RStudio

# Installation

- install R: https://cran.r-project.org/
- install RStudio Desktop: https://posit.co/download/rstudio-desktop/
- let us set the global options properly together

# RStudio panes



# Practice 1

Let's setup our working environment:

1. create a new project
2. create a new R script
3. save the created R script into the project folder directory

**NB:**

- a project is defined by a simple (text) file. Its main benefit is to open RStudio with the correct working directory set (that is, the one where the project file is)
- an R script is a (text) file where we can write R code

# Practice 2

Let's install the following R packages:

1. {tidyverse}
2. {sf}

**NB:**

- I will try to use {} consistently to indicate that something is a package
- installing a package is only required once, unless you want to update the package or unless you installed a new version of R
- I recommend using RStudio for installing packages rather than writing code since you don't want to keep running the installation

# Reading data

# The datasets

Received: 29 April 2025 | Revised: 13 August 2025 | Accepted: 23 September 2025

DOI: 10.1002/ecy.70268

DATA PAPER

ECOLOGY
ECOLOGICAL SOCIETY OF AMERICA

## Data of the Swiss common breeding bird monitoring program

**Nicolas Strebel | Samuel Wechsler | Roman Bühler | Guido Häfliger | Verena Keller | Marc Kéry | Christian Rogenmoser | Martin Spiess | Katarina Varga | Bernard Volet | Niklaus Zbinden | Hans Schmid**

*Since 1999, 267 one-km squares laid out in a mostly systematic grid across all of Switzerland have been surveyed annually by skilled, mostly volunteer ornithologists. Bird populations are recorded using a simplified territory mapping protocol with two visits per square above the timberline and three elsewhere. Surveys are conducted during the breeding season (mid-April to early July) along a square-specific transect route that does not change over the years.*

# Practice 3

1. download the datasets from the following location:
   https://zenodo.org/records/18480886
2. extract the datasets into your R project within a folder called data_MHB

# Practice 3

1. download the datasets from the following location: https://zenodo.org/records/18480886
2. extract the datasets into your R project within a folder called data_MHB

**Pro tips:**

- you can use `getwd()` to check where your project folder is located
- this could be down using R code, but it is not necessary, nor useful here

# Activating the {tidyverse}

```
library(tidyverse)
```

```
-- Attaching core tidyverse packages ------------------------------- tidyverse 2.0.0 --
v dplyr     1.2.1     v readr     2.2.0
v forcats   1.0.1     v stringr   1.6.0
v ggplot2   4.0.3     v tibble    3.3.1
v lubridate 1.9.5     v tidyr     1.3.2
v purrr     1.2.2
-- Conflicts ------------------------------------------------- tidyverse_conflicts() --
x dplyr::filter() masks stats::filter()
x dplyr::lag()    masks stats::lag()
i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to be
```

# The {tidyverse} environment

The {tidyverse} is a meta package encapsulating an ecosystem of R packages (see https://www.tidyverse.org for more).

There are currently 9 core packages automatically attached by the {tidyverse}:


- philosophy: making R more accessible
- backward compatibility is not strictly enforced

# Let's import the first dataset

```
species_count <- read_csv("data_MHB/header-species-count.csv")
```

```
Rows: 7113 Columns: 10
-- Column specification --------------------------------------------------------
Delimiter: ","
dbl (10): samplearea_id, x-koord, y-koord, lon, lat, year, route_length, median_elevat...

i Use `spec()` to retrieve the full column specification for this data.
i Specify the column types or set `show_col_types = FALSE` to quiet this message.
```

# Let's check the first dataset

```
species_count
```

```
# A tibble: 7,113 x 10
  samplearea_id `x-koord` `y-koord`   lon   lat  year route_length median_elevation
          <dbl>     <dbl>     <dbl> <dbl> <dbl> <dbl>        <dbl>            <dbl>
1           268   2725500   1182500  9.08  46.8  1999         4444             1550
2           268   2725500   1182500  9.08  46.8  2000         4444             1550
3           268   2725500   1182500  9.08  46.8  2001         4444             1550
# i 7,110 more rows
# i 2 more variables: n_visits <dbl>, n_recorded_species <dbl>
```

# Let's check the first dataset

```
species_count
```

```
# A tibble: 7,113 x 10
  samplearea_id `x-koord` `y-koord`   lon   lat  year route_length median_elevation
          <dbl>     <dbl>     <dbl> <dbl> <dbl> <dbl>        <dbl>            <dbl>
1           268   2725500   1182500  9.08  46.8  1999         4444             1550
2           268   2725500   1182500  9.08  46.8  2000         4444             1550
3           268   2725500   1182500  9.08  46.8  2001         4444             1550
# i 7,110 more rows
# i 2 more variables: n_visits <dbl>, n_recorded_species <dbl>
```

**Pro tips:**

- check the output (a tibble) carefully to make sure you read in your data properly
- if your are not using {readr}, the output may not be a tibble and would be better checked using `str()`

# Practice 4

- import the file `header-species-count.csv` to obtain the second dataset

```
territories
```

```
# A tibble: 1,152,306 x 10
  species_id euring_id nameeg       namelt         samplearea_id  year territories_visit_1
       <dbl>     <dbl> <chr>        <chr>                  <dbl> <dbl>                <dbl>
1         50        70 Little Grebe Tachybaptus r~           268  1999                    0
2         50        70 Little Grebe Tachybaptus r~           268  2000                    0
3         50        70 Little Grebe Tachybaptus r~           268  2001                    0
# i 1,152,303 more rows
# i 3 more variables: territories_visit_2 <dbl>, territories_visit_3 <dbl>,
#   territories_total <dbl>
```

# Practice 4

- import the file `header-species-count.csv` to obtain the second dataset

```
territories
```

```
# A tibble: 1,152,306 x 10
  species_id euring_id nameeg       namelt         samplearea_id  year territories_visit_1
       <dbl>     <dbl> <chr>        <chr>                  <dbl> <dbl>               <dbl>
1         50        70 Little Grebe Tachybaptus r~           268  1999                   0
2         50        70 Little Grebe Tachybaptus r~           268  2000                   0
3         50        70 Little Grebe Tachybaptus r~           268  2001                   0
# i 1,152,303 more rows
# i 3 more variables: territories_visit_2 <dbl>, territories_visit_3 <dbl>,
#   territories_total <dbl>
```

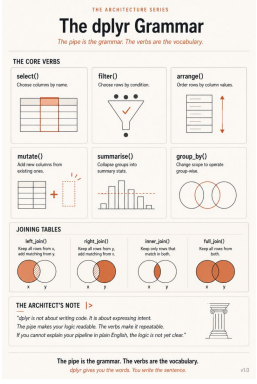
**Pro tips:**

- use `View()` to explore the datasets in an Excel-like fashion

# Wrangling data

# Data wrangling with {dplyr}

In {dplyr} much can be achieved with a few functions




source: https://linkedin.com/in/adalbert-ngongang-phd-637995a3

We will study some of these functions and a few others to get you going

# Choose columns using `select()`

```
species_count |> select(samplearea_id, year, n_recorded_species)
```

```
# A tibble: 7,113 x 3
  samplearea_id  year n_recorded_species
          <dbl> <dbl>              <dbl>
1           268  1999                 27
2           268  2000                 25
3           268  2001                 30
# i 7,110 more rows
```

# Choose columns using `select()`

```
species_count |> select(samplearea_id, year, n_recorded_species)
```

```
# A tibble: 7,113 x 3
  samplearea_id  year n_recorded_species
          <dbl> <dbl>              <dbl>
1           268  1999                 27
2           268  2000                 25
3           268  2001                 30
# i 7,110 more rows
```

**Pro tips:**

- you can use `-column_name` to drop a column called `column_name`
- you can use selection helpers (e.g. `contains()`, `starts_with()`) to select columns using name patterns
- you can use test(s) on column values to select (`where()`)

# A few words about the pipe |>

```
species_count |> select(samplearea_id, year, n_recorded_species)
```

is interpreted exactly as this:

```
select(species_count, samplearea_id, year, n_recorded_species)
```

**NB:**

- the pipe captures what precedes it and considers it as the first argument of the function that follows it
- it makes the code more readable, as it emphasises the sequence of operations applied to the data

# Extract one column using `pull()`

You can use `select()` to choose one column, but it will remain a table (technically an object of class `tibble` or `data.frame`):

```
species_count |> select(n_recorded_species)
```

```
# A tibble: 7,113 x 1
  n_recorded_species
               <dbl>
1                 27
2                 25
3                 30
# i 7,110 more rows
```

Instead `pull()` creates a vector, which can be easier to work with:

```
species_count |> pull(n_recorded_species)
```

```
   [1] 27 25 30 31 29 29 38 27 29 39 30 37 44 36 37 33 35 34 42 31 42 33 37 31 32 33 42 29
  [29] 28 32 28 30 29 26 28 29 31 37 30 27 28 27 30 26 30 27 29 29 29 26 29 28 25 27 37 37
  [57] 42 46 42 41 40 41 40 42 40 45 42 41 44 40 36 34 37 36 39 43 45 41 42 44 43 38 32 34
  [85] 32 37 37 34 34 33 33 33 37 32 34 31 32 32 33 36 35 36 54 39 44 45 42 39 34 37 33 33
 [113] 33 43 41 41 37 34 42 36 39 37 34 37 40 34 38 37 36 37 39 36 31 34 30 39 41 46 42 46
```

# Choose rows using `filter()`

```
species_count |> filter(year == 2025)
```

```
# A tibble: 261 x 10
  samplearea_id `x-koord` `y-koord`   lon   lat  year route_length median_elevation
          <dbl>     <dbl>     <dbl> <dbl> <dbl> <dbl>        <dbl>            <dbl>
1           268   2725500   1182500  9.08  46.8  2025         4444             1550
2           269   2497500   1118500  6.11  46.2  2025         6232              450
3           270   2725500   1214500  9.09  47.1  2025         4081             1350
# i 258 more rows
# i 2 more variables: n_visits <dbl>, n_recorded_species <dbl>
```

# Choose rows using `filter()`

```
species_count |> filter(year == 2025)
```

```
# A tibble: 261 x 10
  samplearea_id `x-koord` `y-koord`   lon   lat  year route_length median_elevation
          <dbl>     <dbl>     <dbl> <dbl> <dbl> <dbl>        <dbl>            <dbl>
1           268   2725500   1182500  9.08  46.8  2025         4444             1550
2           269   2497500   1118500  6.11  46.2  2025         6232              450
3           270   2725500   1214500  9.09  47.1  2025         4081             1350
# i 258 more rows
# i 2 more variables: n_visits <dbl>, n_recorded_species <dbl>
```

**Pro tips:**

- use `%in%` to filter multiple rows (e.g. `year %in% c(2000, 2025)`)
- use `!=` to negate equality (e.g. `year != 2025` to keep all be 2025)

# Use `filter() + if_any() + is.na()` to look for missing values

```
species_count |> filter(if_any(everything(), is.na)) #everything() selects all columns
```

```
# A tibble: 0 x 10
# i 10 variables: samplearea_id <dbl>, x-koord <dbl>, y-koord <dbl>, lon <dbl>,
#   lat <dbl>, year <dbl>, route_length <dbl>, median_elevation <dbl>, n_visits <dbl>,
#   n_recorded_species <dbl>
```

# Use `filter() + if_any() + is.na()` to look for missing values

```
species_count |> filter(if_any(everything(), is.na)) #everything() selects all columns
```

```
# A tibble: 0 x 10
# i 10 variables: samplearea_id <dbl>, x-koord <dbl>, y-koord <dbl>, lon <dbl>,
#   lat <dbl>, year <dbl>, route_length <dbl>, median_elevation <dbl>, n_visits <dbl>,
#   n_recorded_species <dbl>
```

```
territories |> filter(if_any(everything(), is.na))
```

```
# A tibble: 391,247 x 10
  species_id euring_id nameeg       namelt         samplearea_id  year territories_visit_1
       <dbl>     <dbl> <chr>        <chr>                  <dbl> <dbl>                <dbl>
1         50        70 Little Grebe Tachybaptus r~           271  2001                   NA
2         50        70 Little Grebe Tachybaptus r~           271  2002                   NA
3         50        70 Little Grebe Tachybaptus r~           279  2001                   NA
# i 391,244 more rows
# i 3 more variables: territories_visit_2 <dbl>, territories_visit_3 <dbl>,
#   territories_total <dbl>
```

# Choose rows by position using `slice()`

```
species_count |> slice(1:3) # keeps the first 3 rows
```

```
# A tibble: 3 x 10
  samplearea_id `x-koord` `y-koord`   lon   lat  year route_length median_elevation
          <dbl>     <dbl>     <dbl> <dbl> <dbl> <dbl>        <dbl>            <dbl>
1           268   2725500   1182500  9.08  46.8  1999         4444             1550
2           268   2725500   1182500  9.08  46.8  2000         4444             1550
3           268   2725500   1182500  9.08  46.8  2001         4444             1550
# i 2 more variables: n_visits <dbl>, n_recorded_species <dbl>
```

# Choose rows using `slice_min()` or `slice_max()`

```
species_count |>
    slice_max(n_recorded_species, n = 3) # keeps the top 3 sites by n_recorded_species
```

```
# A tibble: 5 x 10
  samplearea_id `x-koord` `y-koord`   lon   lat  year route_length median_elevation
          <dbl>     <dbl>     <dbl> <dbl> <dbl> <dbl>        <dbl>            <dbl>
1           422   2623500   1174500  7.75  46.7  2020         5065             1350
2           511   2707500   1270500  8.87  47.6  2007         6285              350
3           422   2623500   1174500  7.75  46.7  2019         5065             1350
# i 2 more rows
# i 2 more variables: n_visits <dbl>, n_recorded_species <dbl>
```

# Choose rows using `slice_min()` or `slice_max()`

```
species_count |>
    slice_max(n_recorded_species, n = 3) # keeps the top 3 sites by n_recorded_species
```

```
# A tibble: 5 x 10
  samplearea_id `x-koord` `y-koord`   lon   lat  year route_length median_elevation
          <dbl>     <dbl>     <dbl> <dbl> <dbl> <dbl>        <dbl>            <dbl>
1           422   2623500   1174500  7.75  46.7  2020         5065             1350
2           511   2707500   1270500  8.87  47.6  2007         6285              350
3           422   2623500   1174500  7.75  46.7  2019         5065             1350
# i 2 more rows
# i 2 more variables: n_visits <dbl>, n_recorded_species <dbl>
```

**Pro tips:**

- use `with_ties = FALSE`, to obtain the number of rows you want despite ties
- other slicing functions can be useful (e.g. `slice_head()`, `slice_sample()`)

# Choose different rows using `distinct()`

```
species_count |> distinct(samplearea_id, lon, lat)
```

```
# A tibble: 267 x 3
  samplearea_id   lon   lat
          <dbl> <dbl> <dbl>
1           268  9.08  46.8
2           269  6.11  46.2
3           270  9.09  47.1
# i 264 more rows
```

# Chaining multiple operations using the pipe |>

```
species_count |>
    filter(year == 2025) |>
    select(samplearea_id, year)
```

```
# A tibble: 261 x 2
  samplearea_id  year
          <dbl> <dbl>
1           268  2025
2           269  2025
3           270  2025
# i 258 more rows
```

# Chaining multiple operations using the pipe `|>`

```
species_count |>
    filter(year == 2025) |>
    select(samplearea_id, year)
```

```
# A tibble: 261 x 2
  samplearea_id  year
          <dbl> <dbl>
1           268  2025
2           269  2025
3           270  2025
# i 258 more rows
```

**NB:**

- remember, this is interpreted by R as:
  `select(filter(species_count, year == 2025), samplearea_id, year)`
- when chaining operations, keeping the data alone in the first line and indenting the rest improves the readability

# Store your output into a reusable object using `<-` or `->`

## Conventional way

```
species_count_2025 <- species_count |> filter(year == 2025)
```

## Alternative way

```
species_count |> filter(year == 2025) -> species_count_2025
```

**Pro tips:**

- the conventional way helps to spot which objects are being created in the script
- when using many chained operation using the pipe `|>`, the alternative way lead to a more natural reading of the code

# Sort rows using `arrange()`

```
species_count |> arrange(year)
```

```
# A tibble: 7,113 x 10
  samplearea_id `x-koord` `y-koord`   lon   lat  year route_length median_elevation
          <dbl>     <dbl>     <dbl> <dbl> <dbl> <dbl>        <dbl>            <dbl>
1           268   2725500   1182500  9.08  46.8  1999         4444             1550
2           269   2497500   1118500  6.11  46.2  1999         6232              450
3           270   2725500   1214500  9.09  47.1  1999         4081             1350
# i 7,110 more rows
# i 2 more variables: n_visits <dbl>, n_recorded_species <dbl>
```

# Sort rows using `arrange()`

```
species_count |> arrange(year)
```

```
# A tibble: 7,113 x 10
  samplearea_id `x-koord` `y-koord`   lon   lat  year route_length median_elevation
          <dbl>     <dbl>     <dbl> <dbl> <dbl> <dbl>        <dbl>            <dbl>
1           268   2725500   1182500  9.08  46.8  1999         4444             1550
2           269   2497500   1118500  6.11  46.2  1999         6232              450
3           270   2725500   1214500  9.09  47.1  1999         4081             1350
# i 7,110 more rows
# i 2 more variables: n_visits <dbl>, n_recorded_species <dbl>
```

**Pro tips:**

- use multiple variables for hierarchical sorting
- use `desc()` for sorting by decreasing values (e.g. `desc(year)`)

# Practice 5

Use `select()` and/or `filter()` and/or `slice()` and/or `arrange()` to determine:

1. the largest number of recorded species across all years
2. the largest number of recorded species in 2025

# Create new columns using `mutate()`

```
species_count |>
    select(samplearea_id, route_length, year) |>
    filter(year == 2025) |>
    mutate(short_transect = route_length < mean(route_length))
```

```
# A tibble: 261 x 4
  samplearea_id route_length  year short_transect
          <dbl>        <dbl> <dbl> <lgl>
1           268         4444  2025 TRUE
2           269         6232  2025 FALSE
3           270         4081  2025 TRUE
# i 258 more rows
```

# Create new columns using `mutate()`

```
species_count |>
    select(samplearea_id, route_length, year) |>
    filter(year == 2025) |>
    mutate(short_transect = route_length < mean(route_length))
```

```
# A tibble: 261 x 4
  samplearea_id route_length  year short_transect
          <dbl>        <dbl> <dbl> <lgl>
1           268         4444  2025 TRUE
2           269         6232  2025 FALSE
3           270         4081  2025 TRUE
# i 258 more rows
```

**Pro tips:**

- use `.before =` or `.after =` to control where the new column is added

# Collapse groups into summary statistics using `summarise()`

```
species_count |>
    summarise(min_route_length = min(route_length),
              max_route_length = max(route_length))
```

```
# A tibble: 1 x 2
  min_route_length max_route_length
             <dbl>            <dbl>
1             1181             9411
```

# Grouping operations

```
species_count |>
    summarise(max_sp = max(n_recorded_species), .by = year)
```

```
# A tibble: 27 x 2
   year max_sp
  <dbl>  <dbl>
1  1999     53
2  2000     58
3  2001     59
# i 24 more rows
```

# Grouping operations

```
species_count |>
    summarise(max_sp = max(n_recorded_species), .by = year)
```

```
# A tibble: 27 x 2
   year max_sp
  <dbl>  <dbl>
1  1999     53
2  2000     58
3  2001     59
# i 24 more rows
```

**Pro tips:**

- `.by =` works with the other verbs too, but is sometimes called `by` (e.g. in `slice()`)
- `.by =` is safer than `group_by()` which externalises the grouping

# Collapse groups into list-columns using `summarise() + list()`

```
territories |>
    filter(territories_total > 0) |>
    summarise(sp = list(nameeg), .by = c(samplearea_id, year))
```

```
# A tibble: 7,113 x 3
  samplearea_id  year sp
          <dbl> <dbl> <list>
1           279  2001 <chr [44]>
2           279  2002 <chr [46]>
3           279  2006 <chr [47]>
# i 7,110 more rows
```

# Collapse groups into list-columns using `summarise() + list()`

```
territories |>
    filter(territories_total > 0) |>
    summarise(sp = list(nameeg), .by = c(samplearea_id, year))
```

```
# A tibble: 7,113 x 3
  samplearea_id  year sp
          <dbl> <dbl> <list>
1           279  2001 <chr [44]>
2           279  2002 <chr [46]>
3           279  2006 <chr [47]>
# i 7,110 more rows
```

**NB:**

- the column `sp` is called a list-column

# Line-by-line operations using `rowwise()`

```
territories |>
    filter(territories_total > 0) |>
    summarise(sp = list(nameeg), .by = c(samplearea_id, year)) |>
    rowwise() |>
    mutate(mute_swan = "Mute Swan" %in% sp) |>
    ungroup()
```

```
# A tibble: 7,113 x 4
  samplearea_id  year sp         mute_swan
          <dbl> <dbl> <list>     <lgl>
1           279  2001 <chr [44]> FALSE
2           279  2002 <chr [46]> FALSE
3           279  2006 <chr [47]> FALSE
# i 7,110 more rows
```

# Line-by-line operations using `rowwise()`

```
territories |>
    filter(territories_total > 0) |>
    summarise(sp = list(nameeg), .by = c(samplearea_id, year)) |>
    rowwise() |>
    mutate(mute_swan = "Mute Swan" %in% sp) |>
    ungroup()
```

```
# A tibble: 7,113 x 4
  samplearea_id  year sp         mute_swan
          <dbl> <dbl> <list>     <lgl>
1           279  2001 <chr [44]> FALSE
2           279  2002 <chr [46]> FALSE
3           279  2006 <chr [47]> FALSE
# i 7,110 more rows
```

**Pro tips:**

- do not forget to call `ungroup()`, otherwise future calls with also be executed line by line, which can impact the results and be slow

# Counting with `count()`

```
species_count |> count(year)
```

```
# A tibble: 27 x 2
   year     n
  <dbl> <int>
1  1999   241
2  2000   264
3  2001   259
# i 24 more rows
```

# Counting with `count()`

```
species_count |> count(year)
```

```
# A tibble: 27 x 2
   year     n
  <dbl> <int>
1  1999   241
2  2000   264
3  2001   259
# i 24 more rows
```

**Pro tips:**

- you can use multiple variables here to obtain the number of rows for each distinct combination of values across those variables (as in `distinct()`)
- you can count using `summarise()` but `count()` is more direct
- you can sort the outcome (without `arrange()`) using the option `sort = TRUE`

# Combining tables using `left_join()`

```
territories |> left_join(species_count)
```

```
Joining with `by = join_by(samplearea_id, year)`
```

```
# A tibble: 1,152,306 x 18
  species_id euring_id nameeg       namelt          samplearea_id  year territories_visit_1
       <dbl>     <dbl> <chr>        <chr>                   <dbl> <dbl>                <dbl>
1         50        70 Little Grebe Tachybaptus r~            268  1999                    0
2         50        70 Little Grebe Tachybaptus r~            268  2000                    0
3         50        70 Little Grebe Tachybaptus r~            268  2001                    0
# i 1,152,303 more rows
# i 11 more variables: territories_visit_2 <dbl>, territories_visit_3 <dbl>,
#   territories_total <dbl>, `x-koord` <dbl>, `y-koord` <dbl>, lon <dbl>, lat <dbl>,
#   route_length <dbl>, median_elevation <dbl>, n_visits <dbl>, n_recorded_species <dbl>
```

# Combining tables using `left_join()`

```
territories |> left_join(species_count)
```

```
Joining with `by = join_by(samplearea_id, year)`
```

```
# A tibble: 1,152,306 x 18
  species_id euring_id nameeg       namelt          samplearea_id  year territories_visit_1
       <dbl>     <dbl> <chr>        <chr>                   <dbl> <dbl>                <dbl>
1         50        70 Little Grebe Tachybaptus r~            268  1999                    0
2         50        70 Little Grebe Tachybaptus r~            268  2000                    0
3         50        70 Little Grebe Tachybaptus r~            268  2001                    0
# i 1,152,303 more rows
# i 11 more variables: territories_visit_2 <dbl>, territories_visit_3 <dbl>,
#   territories_total <dbl>, `x-koord` <dbl>, `y-koord` <dbl>, lon <dbl>, lat <dbl>,
#   route_length <dbl>, median_elevation <dbl>, n_visits <dbl>, n_recorded_species <dbl>
```

**NB:**

- for `left_join()` to work automatically, some columns with identical names must exist in both datasets (called keys)

# Practice 6

Use the functions we have seen to obtain:

1. the range of median elevation according to the number of visits
2. the 3 most recorded species for each year
3. the number of observations for *Sturnus vulgaris* for each year

# Reshaping data from long to wide format using `pivot_wider()`

Let's create an "site x species" detection table for 2025:

```
territories |>
    filter(year == 2025) |>
    select(samplearea_id, nameeg, territories_total) |>
    pivot_wider(names_from = nameeg,
                values_from = territories_total)
```

```
# A tibble: 261 x 163
  samplearea_id `Little Grebe` `Great Crested Grebe` `Grey Heron` `Little Bittern`
          <dbl>          <dbl>                 <dbl>        <dbl>             <dbl>
1           268              0                     0            0                 0
2           269              0                     0            0                 0
3           270              0                     0            0                 0
# i 258 more rows
# i 158 more variables: `White Stork` <dbl>, `Mute Swan` <dbl>, `Greylag Goose` <dbl>,
#   `Egyptian Goose` <dbl>, Mallard <dbl>, `Red-crested Pochard` <dbl>,
#   `Tufted Duck` <dbl>, `Common Merganser` <dbl>, `European Honey Buzzard` <dbl>,
#   `Red Kite` <dbl>, `Black Kite` <dbl>, `Eurasian Goshawk` <dbl>,
#   `Eurasian Sparrowhawk` <dbl>, `Common Buzzard` <dbl>, `Eurasian Hobby` <dbl>,
#   `Common Kestrel` <dbl>, `Black Grouse` <dbl>, `Rock Ptarmigan` <dbl>, ...
```

# Practice 7

Try to use `pivot_longer()` to get back to the original long format of the data from the output of the following code:

```
territories |>
    filter(year == 2025) |>
    select(samplearea_id, nameeg, territories_total) |>
    pivot_wider(names_from = nameeg,
                values_from = territories_total)
```

# Wrangling spatial data

# The simple feature package {sf}

{sf} is a package developed to handle spatial data which plays along with the {tidyverse} (it is inspired by PostGIS, see e.g. postgis.net/workshops/postgis-intro/index.html)

```
library(sf)
```

```
Linking to GEOS 3.14.1, GDAL 3.12.4, PROJ 9.8.1; sf_use_s2() is TRUE
```

```
species_count |>
    st_as_sf(coords = c("lon", "lat"), crs = 4326) -> species_count_sf
```

**NB:**

- the order of the columns provided to `coords =` matters (see next)
- the coordinate reference system (`crs`) is here provided as a so-called EPSG code (4326 = WGS 84)

# Simple feature collections

```
species_count_sf
```

```
Simple feature collection with 7113 features and 8 fields
Geometry type: POINT
Dimension:     XY
Bounding box:  xmin: 6.110458 ymin: 45.8476 xmax: 10.40901 ymax: 47.72377
Geodetic CRS:  WGS 84
# A tibble: 7,113 x 9
  samplearea_id `x-koord` `y-koord`  year route_length median_elevation n_visits
*         <dbl>     <dbl>     <dbl> <dbl>        <dbl>            <dbl>    <dbl>
1           268   2725500   1182500  1999         4444             1550        3
2           268   2725500   1182500  2000         4444             1550        3
3           268   2725500   1182500  2001         4444             1550        3
# i 7,110 more rows
# i 2 more variables: n_recorded_species <dbl>, geometry <POINT [°]>
```

**Pro tips:**

- check carefully the bounding box to make sure that latitudes and longitudes are OK
- simple feature collections are tibbles with a specific list-column called `geometry`

# Computing the total surface area

```
species_count_sf |>
    summarise(geometry = st_combine(geometry)) |> # all points are brought together
    mutate(hull = st_convex_hull(geometry), # create shapes encompassing all points
           area_m2 = st_area(hull), # compute the area in m^2
           area_km2 = units::set_units(area_m2, "km^2")) |> # express area in km^2
    select(-hull) |> # drop the hull
    st_drop_geometry() # drop the geometry
```

```
# A tibble: 1 x 2
       area_m2 area_km2
*        [m^2]   [km^2]
1 48995663682.   48996.
```

# Computing the total surface area

```
species_count_sf |>
    summarise(geometry = st_combine(geometry)) |> # all points are brought together
    mutate(hull = st_convex_hull(geometry), # create shapes encompassing all points
           area_m2 = st_area(hull), # compute the area in m^2
           area_km2 = units::set_units(area_m2, "km^2")) |> # express area in km^2
    select(-hull) |> # drop the hull
    st_drop_geometry() # drop the geometry
```

```
# A tibble: 1 x 2
       area_m2 area_km2
*        [m^2]    [km^2]
1 48995663682.   48996.
```

- the syntax `namespace::function` allows one to use a function from a package without calling `library()`
- you must use `st_drop_geometry()` to drop the geometry column from a sf collection; `select()` won't work because `geometry` is a protected column

# Computing the distance of each site to the center of the studied area

```
species_count_sf |>
  distinct(samplearea_id, geometry) |>
  mutate(centroid = st_centroid(st_combine(geometry)),
         dist = st_distance(centroid, geometry, by_element = TRUE))
```

```
Simple feature collection with 267 features and 2 fields
Active geometry column: geometry
Geometry type: POINT
Dimension:     XY
Bounding box:  xmin: 6.110458 ymin: 45.8476 xmax: 10.40901 ymax: 47.72377
Geodetic CRS:  WGS 84
# A tibble: 267 x 4
  samplearea_id            geometry            centroid    dist
*         <dbl>         <POINT [°]>         <POINT [°]>     [m]
1           268 (9.082159 46.78185) (8.234991 46.79076)  64509.
2           269 (6.110458 46.21013) (8.234991 46.79076) 174955.
3           270 (9.090984 47.06963) (8.234991 46.79076)  72016.
# i 264 more rows
```

**NB:** if a computation seems to take a very long time, it is *often* a sign that you are not handling the problem properly

# Practice 8

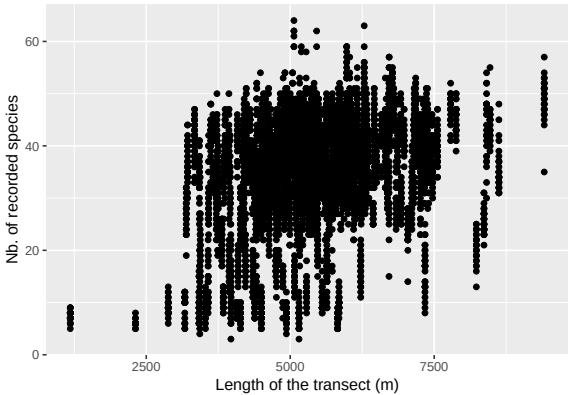
Use the functions we have seen to obtain:

1. the sampled area (sensu convex hull) across years
2. the IDs of the sampled areas which are most peripheral

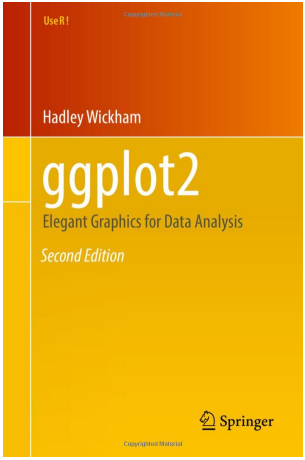
# Plotting data

# Plots are built in {ggplot2} by adding components together

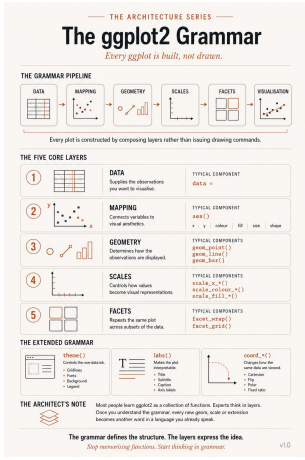
```
ggplot(species_count) +
  geom_point(aes(y = n_recorded_species, x = route_length)) +
  labs(y = "Nb. of recorded species", x = "Length of the transect (m)")
```



# The grammar of graphics plotting framework



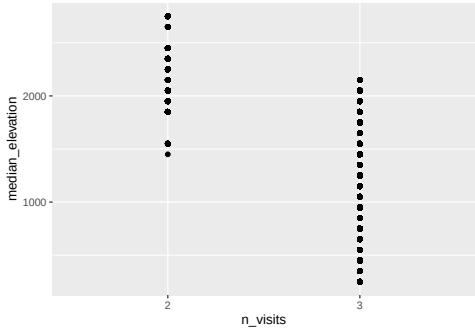

source: https://ggplot2-book.org




source: https://linkedin.com/in/adalbert-ngongang-phd-637995a3

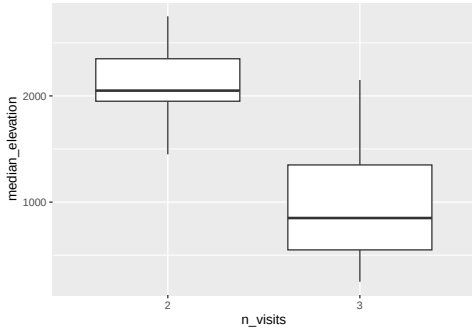
# Geoms are the fundamental building blocks

```
species_count |>
  mutate(n_visits = factor(n_visits)) |>
  ggplot() +
    geom_point(aes(y = median_elevation, x = n_visits))
```



# Geoms are the fundamental building blocks

```
species_count |>
  mutate(n_visits = factor(n_visits)) |>
  ggplot() +
    geom_boxplot(aes(y = median_elevation, x = n_visits))
```

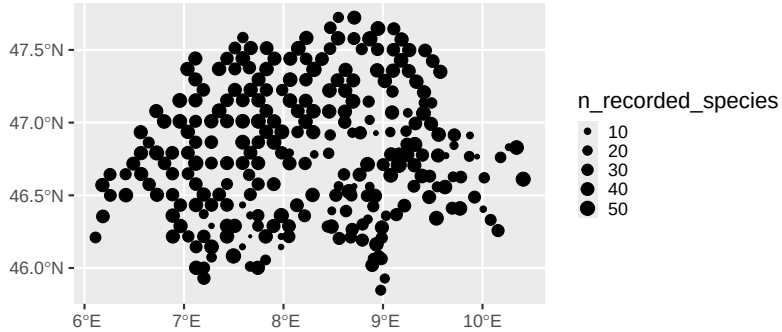


# Practice 9

1. check the possible geoms in the official ggplot2 cheat sheet (accessible via Help)
2. explore briefly r-graph-gallery.com

# Objects created with {sf} can be plotted using {ggplot2} too

```
species_count_sf |>
  filter(year == 2025) |>
  ggplot() +
    geom_sf(aes(size = n_recorded_species)) # geometry = geometry is assumed
```



# Understanding aesthetic mappings

- `aes()` is used to defined how data connect to plotting elements via aesthetics
- different geom expect different mandatory and optional aesthetics
  (see e.g. `?geom_point`)

# Understanding aesthetic mappings

- `aes()` is used to defined how data connect to plotting elements via aesthetics
- different geom expect different mandatory and optional aesthetics (see e.g. `?geom_point`)

- `aes()` is not the place to force colours, sizes, etc.; it only forces relationship:

## Bad

```
ggplot(species_count) +
  geom_point(aes(y = n_recorded_species, x = route_length, colour = "blue"))
```

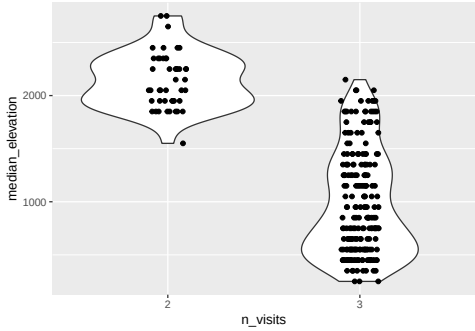
## Good

```
ggplot(species_count) +
  geom_point(aes(y = n_recorded_species, x = route_length), colour = "blue")
```

# Understanding aesthetic mappings

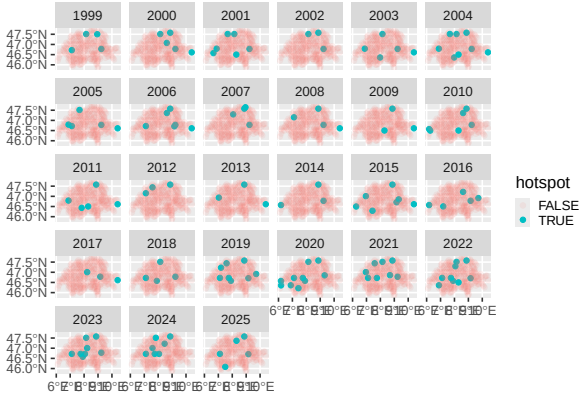
- `aes()` can be located outside a geom to apply to several of them at once:

```
species_count |>
    filter(year == 2025) |>
    mutate(n_visits = factor(n_visits)) |>
    ggplot(aes(y = median_elevation, x = n_visits)) +
      geom_violin() +
      geom_jitter(height = 0, width = 0.1)
```



# Plot different subsets using faceting

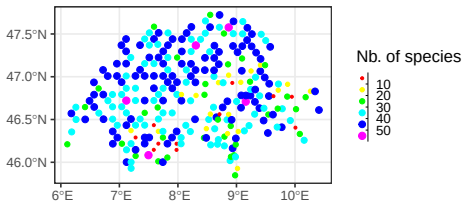
```
species_count_sf |>
    mutate(hotspot = factor(n_recorded_species > 50)) |>
    ggplot() +
      geom_sf(aes(colour = hotspot, alpha = hotspot)) +
      facet_wrap(~ year)
```



# Making plots beautiful & readable

Much can be achieved by using a better theme and defining scales:

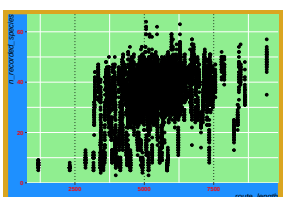
```
species_count_sf |>
    filter(year == 2025) |>
    ggplot() +
        geom_sf(aes(size = n_recorded_species,
                    colour = n_recorded_species)) +
        scale_color_binned(palette = rainbow, name = "Nb. of species") +
        scale_size_binned(range = c(0.5, 5), name = "Nb. of species") +
        guides(size = guide_bins(), color = guide_bins()) + # unusually needed here
        theme_bw(base_size = 25)
```



# Making plots beautiful & readable

To modify the theme yourself, use `theme_sub_*()` or ( `theme()` ) in combination with the appropriate `element_*()` function:

```
ggplot(species_count) +
  geom_point(aes(y = n_recorded_species, x = route_length)) +
  theme_sub_axis(text = element_text(size = 15, face = "bold", colour = "red"),
                 title = element_text(face = "italic", hjust = 1)) +
  theme_sub_panel(grid.major.x = element_line(linetype = "dotted", colour = "black"),
                  background = element_rect(fill = "lightgreen")) +
  theme_sub_plot(background = element_rect(fill = "dodgerblue", colour = "goldenrod",
                                           linewidth = 10))
```

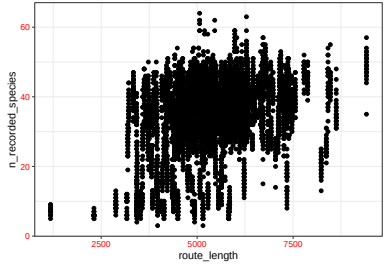


# Making plots beautiful & readable

You can define a common theme to be used for all your plots:

```
(theme_bw(base_size = 20) +
  theme_sub_axis(text = element_text(colour = "red"))) |>
  theme_set()

ggplot(species_count) +
  geom_point(aes(y = n_recorded_species, x = route_length))
```




**NB:** the extra `()` are needed since `|>` binds more tightly than `+`

# Practice 10

- combine data wrangling and plotting to illustrate the change in species abundance across years for the top 6 most encountered species

# A dash of base R

# What is base R?

this is not base R (it relies on {tidyverse}, an external R package)

```
species_count |>
    summarise(mean_nb_sp = mean(n_recorded_species), .by = year)
```

```
# A tibble: 27 x 2
   year mean_nb_sp
  <dbl>      <dbl>
1  1999       31.4
2  2000       32.5
3  2001       32.7
# i 24 more rows
```

# What is base R?

## this is typical base R (it does not rely on an external R package)

```
results <- data.frame(year = unique(species_count$year), mean_nb_sp = NA)
for (y in unique(species_count$year)) {
    n_recorded_species_year <- species_count$n_recorded_species[species_count$year == y]
    results[results$year == y, "mean_nb_sp"] <- mean(n_recorded_species_year)
}
head(results, n = 3)
```

```
  year mean_nb_sp
1 1999   31.36515
2 2000   32.48485
3 2001   32.70270
```

**NB:**

- it looks more complicated (but from a developer's perspective, it is actually clearer)

# What is base R?

but this is also base R (and a little closer to {tidyverse} in spirit)

```
results <- with(species_count,
                aggregate(list(mean_nb_sp = n_recorded_species),
                          by = list(year = year), FUN = mean))
head(results, n = 3)
```

```
  year mean_nb_sp
1 1999   31.36515
2 2000   32.48485
3 2001   32.70270
```

**NB:**

- there are often plenty of alternatives to achieve the same output

# Extracting elements from vectors

```
some_vector <- letters
some_vector
```

```
 [1] "a" "b" "c" "d" "e" "f" "g" "h" "i" "j" "k" "l" "m" "n" "o" "p" "q" "r" "s" "t" "u"
[22] "v" "w" "x" "y" "z"
```

```
some_vector[c(1, 12, 5, 24)]
```

```
[1] "a" "l" "e" "x"
```

```
some_vector[some_vector %in% c("a", "l", "e", "x")]
```

```
[1] "a" "e" "l" "x"
```

```
some_namedvector <- c(a = 1, b = 5)
some_namedvector["b"]
```

```
b
5
```

# Extracting elements from lists

```
some_list <- list(1, 1:5)
some_list
```

```
[[1]]
[1] 1

[[2]]
[1] 1 2 3 4 5
```

```
some_list[2]
```

```
[[1]]
[1] 1 2 3 4 5
```

```
some_list[[2]]
```

```
[1] 1 2 3 4 5
```

```
some_list[[2]][2]
```

```
[1] 2
```

# Extracting elements from named lists

```
some_namedlist <- list(a = 1, b = 1:5)
some_namedlist["b"]
```

```
$b
[1] 1 2 3 4 5
```

```
some_namedlist[["b"]]
```

```
[1] 1 2 3 4 5
```

```
some_namedlist$b
```

```
[1] 1 2 3 4 5
```

# Extracting elements from matrices

```
some_matrix <- matrix(1:9, nrow = 3)
some_matrix
```

```
     [,1] [,2] [,3]
[1,]    1    4    7
[2,]    2    5    8
[3,]    3    6    9
```

```
some_matrix[2, ]
```

```
[1] 2 5 8
```

```
some_matrix[, 2]
```

```
[1] 4 5 6
```

```
some_matrix[2, 2]
```

```
[1] 5
```

# Extracting elements from matrices

```
some_matrix[c(1, 3), c(1, 3)]
```

```
     [,1] [,2]
[1,]    1    7
[2,]    3    9
```

```
some_matrix[cbind(c(1, 3), c(1, 3))]
```

```
[1] 1 9
```

# Extracting elements from named matrices

```
colnames(some_matrix) <- paste0("col_", 1:3)
rownames(some_matrix) <- paste0("row_", 1:3)
some_matrix
```

```
      col_1 col_2 col_3
row_1     1     4     7
row_2     2     5     8
row_3     3     6     9
```

```
some_matrix["row_2", "col_3"]
```

```
[1] 8
```

# Extracting elements from data.frames / tibbles

```
species_count[1, ]
```

```
# A tibble: 1 x 10
  samplearea_id `x-koord` `y-koord`   lon   lat  year route_length median_elevation
          <dbl>     <dbl>     <dbl> <dbl> <dbl> <dbl>        <dbl>            <dbl>
1           268   2725500   1182500  9.08  46.8  1999         4444             1550
# i 2 more variables: n_visits <dbl>, n_recorded_species <dbl>
```

```
species_count[, 1]
```

```
# A tibble: 7,113 x 1
  samplearea_id
          <dbl>
1           268
2           268
3           268
# i 7,110 more rows
```

# Extracting elements from data.frames / tibbles

```
species_count[, "year"]
```

```
# A tibble: 7,113 x 1
   year
  <dbl>
1  1999
2  2000
3  2001
# i 7,110 more rows
```

```
species_count["year"]
```

```
# A tibble: 7,113 x 1
   year
  <dbl>
1  1999
2  2000
3  2001
# i 7,110 more rows
```

# Extracting elements from data.frames / tibbles

```
species_count_small <- slice(species_count, 1:10) # for smaller data
species_count_small[["year"]]
```

```
 [1] 1999 2000 2001 2002 2003 2004 2005 2006 2007 2008
```

```
species_count_small$year
```

```
 [1] 1999 2000 2001 2002 2003 2004 2005 2006 2007 2008
```

# A few very useful base R functions I would be sad to work without

Check the following on your own time:

- `help()` for help (or `?` for short)
- `plot()` for fast & dirty plots
- `str()` for inspecting objects
- `dput()` for sharing a reproducible example when asking for help (useful for LLMs)
- `function()` for creating a function (or `\()` for short)
- `debugonce()` for debugging a function line by line
- `rnorm()`, `runif()` for simulating data
- `set.seed()` for "fixing" randomness
- `replicate()` for doing something several times

**Pro tips:**

- check them out in your own time; they are worth learning about

# House keeping

# When and what to update

- update R packages regularly using the RStudio “Packages tab” (at least once a month)
- update R a few times a year:
  check www.r-project.org (May and October tend to be good times)
- update RStudio at least after each update of R

# Learning more

# Getting help

- don't forget that functions have help pages associated to them
- R for Data Science free book: r4ds.hadley.nz
- there is an active Slack channel: dslc.io
- talk to each others
- LLMs?
  if you do, don't blindly copy/paste, and beware:
  LLMs make mistakes and make things up

# Getting better

- learn “base R” too (e.g. cran.r-project.org/doc/manuals/r-release/R-intro.html)
- try to understand why things do not behave as you expect by experimenting
  avoid to always use a workaround
- do use LLMs to review your code (rather than to create it)
  the more you know, the more LLMs will be helpful to you!

Questions?